# Security Regression Testing of Indirect Prompt Injection Defenses under Versioned Agent Tool Contracts

Nguyễn Thùy Dương, Đặng Ngọc Huy, and Vũ Hồng Quang

Group 11
Cybersecurity Governance Policy Course

**Abstract.** LLM agents can invoke tools to read and modify data. Documents or tool outputs may contain untrusted instructions that redirect an agent beyond the user's task. This report presents a security regression-testing design for a tool contract migrating from version $v0$ to $v1$. The design holds the task, initial state, and agent configuration constant while measuring security and task utility as separate outcomes. The group implements task-scoped file authorization under frozen and explicitly migrated policy states and checks the environment before and after tool calls. Deterministic source checks show that the frozen policy rejects the new argument structure, while the migrated policy completes the authorized action. These checks validate an implementation slice; they do not establish resistance to prompt injection by a real language model. The report covers background, related work, the proposed solution, reproducibility, limitations, and a plan for further evaluation.

**Keywords:** LLM agents · indirect prompt injection · tool contracts · security regression · access control

## 1 Problem Statement

### 1.1 Context and Motivation

An LLM agent commonly follows a three-step loop: it reads a request and data, proposes a tool call, and uses the tool result to decide what to do next. Unlike a text-only system, an agent may send email, create calendar events, or modify files. A reasoning error can therefore change the environment.

Indirect prompt injection occurs when a third party places instructions in data the agent is allowed to read, rather than in the user's direct request. For example, a work document may ask the agent to perform an action outside its assigned task. The security boundary is action authorization: reading an instruction does not grant permission to carry it out [3,2].

Systems must also handle tool-interface changes. A policy may check a field in an older argument structure but fail to interpret the newer structure correctly. A schema-valid change is not necessarily authorization-equivalent. Regression testing must measure both forbidden effects and completion of legitimate tasks.

### 1.2 Research Question and Objectives

The research question is whether a defense that prevents a harmful effect under tool contract $v0$ retains that protection under $v1$, and whether an explicit policy migration repairs any mismatch without losing task utility. For this course project, the group defines a measurement design and implements part of the proposed solution to check feasibility.

The report has three technical objectives: define a paired comparison, implement a task-scoped authorization guard, and independently inspect post-execution state. The fixed research topic is *Security Regression Testing of Indirect Prompt Injection Defenses under Versioned Agent Tool Contracts*. This report does not claim a new defense mechanism or population-level evidence of effectiveness.

### 1.3 Scope and Threat Model

The attacker may control email, document, or tool-output content read by the agent. The attacker cannot modify the trusted policy, dispatcher, tool implementation, or oracle. Experiments use a sandbox with synthetic data. Model poisoning, compromise of trusted Python code, and end-to-end information-flow guarantees across multiple agents are outside scope. These assumptions limit the interpretation of results: a dispatcher guard cannot protect a deliberately bypassed execution path.

## 2 Background

### 2.1 Tool Contracts and Task-Scoped Authorization

A tool contract specifies operation names, argument schemas, defaults, resource scope, and effect semantics. A rename may preserve meaning, while changing how a resource or destination is represented can alter the inputs to authorization. This report distinguishes the two so that not every interface change is labeled a security regression.

Least privilege grants an agent only the operations needed for its task. A capability may constrain operations, resources, destinations, expiration, and call counts. The implementation in this report covers tools and resources for one task; expiration, quotas, and approval are possible extensions, not verified features.

### 2.2 Pre-Execution Enforcement and Post-Execution Measurement

The guard checks a call before it reaches the executor. ALLOW dispatches the tool call; DENY returns a policy response without executing the tool. If the authorization structure cannot be interpreted, the guard rejects the call, a behavior known as *fail closed*. This can preserve security while reducing utility.

An oracle is an independent scoring function. The security oracle reads pre- and post-execution state to identify forbidden effects; the utility oracle checks whether the task was completed. ALLOW does not prove task success, and DENY alone does not prove that no effect occurred. A tool error is not a security success. Execution errors, policy decisions, and oracle outcomes must therefore be recorded separately.

### 2.3 Paired Comparison and Missing Data

Let $H_{i,v} \in \{0, 1\}$ denote the harmful effect for case $i$ under version $v$. A security regression occurs when $H_{i,v0} = 0$ and $H_{i,v1} = 1$. All four transitions, 00, 01, 10, and 11, must be retained; reporting only the attack-success rate under $v1$ can hide the distinction between regressions and repairs.

$$\widehat{\Delta}_{\text{sec}} = \frac{1}{n} \sum_{i=1}^{n} (H_{i,v1} - H_{i,v0}). \tag{1}$$

Here, $n$ is the number of valid scored pairs. Pairs with missing evidence remain in the planned denominator and are reported as unscorable; they are not replaced with zero. For utility $U$, a regression is $U_{v0} = 1$ and $U_{v1} = 0$. The two endpoints have different interpretations.

Variants derived from the same source task are not independent observations. A later study must cluster them by source task when estimating uncertainty. This course report does not draw statistical inferences from deterministic checks.

## 3 Related Work

AgentDojo provides a stateful environment for evaluating agents under prompt injection and measures both security and utility [2]. This project uses it as a source of tasks and tools. A successful call to a simulated tool verifies integration only; agent evaluation requires actions generated by the model.

InjecAgent studies indirect prompt injection in tool-integrated agents [5]. It motivates distinguishing attack instructions in content from actions produced by the agent. This report does not treat a handwritten call sequence as equivalent to an episode in which a model reads a carrier and decides what to do.

CaMeL separates untrusted data from control flow and uses capability checks [1]. Progent focuses on privilege control through policy [4]. Because authorization and runtime enforcement are established research topics, TaskFileScope serves here as a reference component for testing rather than a claim of a new protection principle.

The project focuses on measuring the same defense under two tool contracts. This is a testable evaluation question. The works summarized here are selected close references, not a complete systematic review, and do not establish the novelty of the research contribution.

## 4 Proposed Solution and Implemented Component

### 4.1 Architecture and Execution Flow

Trusted inputs consist of the user request, task policy, contract version, and experiment manifest. The agent receives tool data and proposes an action. The dispatcher sends the action through the guard; only an authorized call reaches the executor. Each call records the policy identity, argument digest, decision, dispatch status, and pre/post state hashes.

At the end of an episode, an independent oracle scores utility or harmful effects from the retained evidence.

The manifest fixes the planned cases before execution. The capture bridge starts a fresh process to reduce state leakage between episodes and records the runtime version and requests/responses. The validator checks the evidence, and the statistics adapter accepts only valid pairs. Hashes detect changes and support consistency checks; they do not authenticate that a provider generated a response.

### 4.2 TaskFileScope and Policy Migration

The implemented slice uses Workspace task `user_task_29`: read the team-activities file and append activities to it. The trusted policy grants only filename search for the specified file, reading file ID 3, and appending to file ID 3. Other tools are denied. The policy does not derive permissions from document content, attack labels, or model verdicts.

Under $v0$, the read and append tools use a top-level `file_id`. Under $v1$, the same tools receive a `resource` object containing `space` and `file_id`; the space must be `cloud_drive`. The adapter validates the structure and maps it to the corresponding native argument. There is no automatic fallback to the old field.

Both configurations belong to the same defense family. F keeps the $v0$ policy unchanged under both contract versions. M uses the same $v0$ policy under $v0$ and changes only the argument decoder to $v1$ under contract $v1$. Tool and resource grants do not change. F and M see the same tool schema within each contract version.

### 4.3 Enforcement Rules and Evidence

The guard rejects ungranted tools, extra arguments, invalid types, incorrect scope, or a different file. DENY is a normal policy response; the native tool is not called. Native tool failures remain execution errors. For a denial with typed state, the pre/post hashes must match. The raw trace records what the agent proposed; the canonical trace normalizes arguments for the upstream scorer.

Filename search retains AgentDojo's substring semantics. This guard is not a system-wide read-confidentiality mechanism. It also does not protect against malicious Python code that can access the callable directly within the process.

### 4.4 Relevance to Cybersecurity Governance

A schema change should be governed like a policy change: assign an owner to review permissions, test before deployment, and retain versioned evidence. If a policy has not been migrated, an explicit denial is clearer than guessing the intended meaning. Organizations must also monitor legitimate tasks that are blocked, because denying every tool call does not meet operational needs.

# 5 Implementation and Evaluation Plan

## 5.1 Environment and Artifact Organization

The implementation is written in Python and uses AgentDojo Workspace v1.2.2 at commit `089ed468cf3e`. The historical reproduction environment pins CPython 3.12.13, Pydantic 2.11.4, and PyYAML 6.0.2. Existing state fixtures also depend on the Windows cp1252 default encoding with Python UTF-8 mode disabled. This is a reproduction constraint for the current data, not a recommendation to use cp1252 for new data.

The guard and migration adapter are in the TaskFileScope module; the source preflight runs the authorized operation sequence through ToolsExecutor. The historical name-only control remains in the v2 manifest and validator. The TaskFileScope amendment adds the resource schema; its report acceptance path is being strengthened and is not a complete research result.

## 5.2 Component Conformance Results

Table 1 reports the observed source-conformance outcomes. The input is a deterministic sequence for the user task, not an action generated by a real LLM. Each cell starts from the same reset source state, and utility is computed by the upstream scorer.

**Table 1.** Utility in four TaskFileScope source checks

| Configuration | Contract | Policy decoder | Utility |
|---|---|---|---|
| F | $v0$ | $v0$ | Success |
| F | $v1$ | $v0$ | Not successful |
| M | $v0$ | $v0$ | Success |
| M | $v1$ | $v1$ | Success |

F/$v1$ does not understand the new resource path and therefore denies append. M/$v1$ interprets the resource correctly and allows append to the authorized file. This matches the predicted utility loss for a frozen policy. It does not demonstrate a security regression: the task grants no deletion permission under either version, so this slice cannot distinguish the primary harmful-effect hypothesis.

An integration check with a fake OpenAI-compatible server ran search and append calls through the fresh capture process: F/$v1$ preserved state when denying the call, while M/$v1$ changed state after append. This is evidence that the execution and evidence-collection path works. A fake server cannot show that a model resists prompt injection.

The regression suite at snapshot `e678ea2` recorded 144 passing tests with no failures or skips. This count applies to that snapshot; it does not validate research changes made afterward. The tests cover metadata, traces, reset behavior, missing evidence, and request retries. Test counts are not a measure of security effectiveness.

### 5.3 Reproduction and Acceptance Criteria

In the repository's pinned environment, run the TaskFileScope source preflight and the corresponding amendment integration tests. The accompanying LaTeX README provides paths and PowerShell commands. Compiling the report does not require the AgentDojo environment or an API key.

Component acceptance requires that unauthorized calls do not reach the native executor; denials do not change typed state; F does not change its policy across contracts; M changes only the declared decoder; and utility is scored independently. Provider failures, reset errors, and missing traces must remain in the planned sample rather than being removed to improve results.

### 5.4 Next Experimental Design

The research phase needs a treatment in which an authorization change can expose a harmful effect. F must not be weakened to manufacture a desired result. For each case, hold the model snapshot, task, carrier, and reset constant, and run F/M under both $v0$ and $v1$. Keep the attack endpoint separate from default-control utility, retain all four transitions, and record reasons for unscorable cases.

With sufficient data, report the paired risk difference, source-task-clustered bootstrap intervals, and McNemar tests where the independence assumptions are appropriate. The research plan calls for at least three model snapshots, held-out carrier checks, and independent reproduction. No dataset meeting these gates is reported here.

## 6 Limitations and Conclusion

The implementation shows how to place task-scoped authorization before tool execution, migrate a resource argument explicitly, and measure utility with an independent scorer. This is a concrete solution component for the course project; it is not a complete empirical study of prompt-injection resistance.

The main limitations are one source task, one group-designed migration, a deterministic action sequence, and a fake provider. The design does not test paraphrases, partial leakage, tools outside the sandbox, or multiple models. Timestamp variation in append operations may also change the post-state hash even when utility is the same. Test counts must not be generalized into real-world system safety rates.

The next technical step is to complete the plan–record–oracle binding and obtain independent review. Real-model data should then be collected under a frozen design. The current conclusion concerns the feasibility of the testing method and a checked implementation slice; claims about security effectiveness require additional evidence.

## References

1. Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., Tramèr, F.: Defeating prompt injections by design. arXiv preprint arXiv:2503.18813 (2025), https://arxiv.org/abs/2503.18813
2. Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramèr, F.: Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In: Advances in Neural Information Processing Systems. vol. 37, pp. 82895–82920 (2024). https://doi.org/10.52202/079017-2636
3. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. arXiv preprint arXiv:2302.12173 (2023), https://arxiv.org/abs/2302.12173
4. Shi, T., He, J., Wang, Z., Wu, L., Li, H., Guo, W., Song, D.: Progent: Programmable privilege control for llm agents. arXiv preprint arXiv:2504.11703 (2025), https://arxiv.org/abs/2504.11703
5. Zhan, Q., Liang, Z., Ying, Z., Kang, D.: Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: Findings of ACL 2024. pp. 10471–10506 (2024). https://doi.org/10.18653/v1/2024.findings-acl.624